Progettazione e realizzazione di un sistema distribuito per il monitoraggio ambientale dei parchi comunali

Ludovico Di Iorio
0336019
ludovico.diiorio@students.uniroma2.eu

I. Abstract

Il presente lavoro descrive la progettazione e la realizzazione di un sistema distribuito, che sfrutta il pattern dei microservizi e dell'edge computing. Per la realizzazione sono state utilizzate le tecnologie di Kubernetes (Minikube e k3s) e i servizi offerti da AWS (EC2 e Dynamic IP), oltre che differenti linguaggi di programmazione (Go, Python, Js).

II. Introduction

Il monitoraggio delle condizioni ambientali dei parchi comunali è fondamentale per le amministrazioni locali che necessitano di dati aggiornati per ottimizzare la manutenzione e garantire la qualità degli spazi verdi urbani.

Il progetto sviluppato presenta un sistema per il monitoraggio ambientale dei parchi comunali del comune di Aprilia (LT) basato su un'architettura edge-cloud. Il sistema è composto quindi da dispositivi edge dotati di sensori per il rilevamento delle condizioni climatiche (temperatura, umidità, luminosità e qualità dell'aria) che raccolgono periodicamente dati ambientali e li trasmettono a un server centralizzato per l'elaborazione e consentirne la visualizzazione.

I dispositivi, essenzialmente, raccolgono dati, tramite dei sensori, per poi sottoporli ad un pre-processamento prima della trasmissione, così da decentralizzare la loro elaborazione. Il servizio di backend è ospitato su un'istanza nel cloud, la quale si occupa anche della gestione di altri servizi (come il frontend server) e utilizza Minikube per gestire tali servizi come microservizi. La persistenza dei dati è gestita con un database relazionale e la visualizzazione dei dati in tempo reale tramite un'interfaccia web.

L'architettura adottata integra efficacemente diversi concetti fondamentali dei sistemi distribuiti come l'edge computing, la containerizzazione e servizi cloud, creando una soluzione scalabile e affidabile per il monitoraggio ambientale urbano.

III. Background

Il progetto si colloca nell'ambito dei sistemi distribuiti e del cloud computing, due paradigmi fondamentali nello sviluppo di applicazioni moderne scalabili e modulari. L'obiettivo è integrare tecnologie edge, microservizi, container e orchestrazione, per simulare un ambiente realistico in cui dati ambientali vengono raccolti, pre-elaborati e visualizzati, il tutto tramite un'infrastruttura distribuita.

Uno degli elementi chiave è il concetto di edge computing, che prevede l'elaborazione dei dati direttamente nei dispositivi periferici vicini alla fonte, riducendo la latenza e il traffico verso il server nel cloud. Nel contesto di questo progetto, i dispositivi edge sono simulati da macchine virtuali leggere e gestite tramite k3s, una distribuzione semplificata di Kubernetes adatta a dispositivi con risorse limitate. Ogni dispositivo simula la raccolta dati da sensori ambientali e invia i risultati, in modo intelligente, al server centrale.

La comunicazione tra dispositivi edge e il servizio di backend avviene tramite gRPC, un moderno sistema di Remote Procedure Call sviluppato da Google, che consente la trasmissione efficiente di messaggi binari su HTTP/2. gRPC è stato scelto per la sua leggerezza e per il supporto nativo alla serializzazione tramite Protocol Buffers, ideale per scenari IoT e real-time.

Il progetto ha un'architettura basata su microservizi, la quale garantisce scalabilità, resilienza e agilità di sviluppo. Questa struttura permette di gestire meglio la complessità, accelerare il ciclo di rilascio e isolare i fallimenti, garantendo che il malfunzionamento di un servizio non comprometta l'intera applicazione.

Per gestire l'orchestrazione e il ciclo di vita dei microservizi, abbiamo scelto di utilizzare Minikube. All'interno di una istanza EC2 di

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

AWS, è ospitato il cluster Minikube con i microservizi per il backend e per il frontend, tutti containerizzati tramite Docker.

Per la persistenza dei dati è stato scelto MariaDB, un database relazionale compatibile con MySQL, eseguito in un container Docker su un'istanza EC2 separata. Questa architettura distribuita consente una separazione delle responsabilità e una maggiore flessibilità gestionale.

IV. SYSTEM ARCHITECTURE

Il sistema è composto da cinque componenti:

A. Dispositivi edge

I dispositivi edge sono dei dispositivi che si occupano di raccogliere dati ambientali del parco a cui sono stati assegnati, pre-processarli, ed infine inviarli al server nel cloud se superano i controlli di validità. I dispositivi edge comunicano solamente con il servizio di backend e lo fa tramite gRPC.
L'utilizzo di dispositivi edge consente di avere diversi vantaggi:

- *riduzione della latenza e un miglioramento della reattività:* il pre-processamento dei dati permette di ridurre il numero di messaggi scambiati con il servizio di backend nel cloud;
- *maggiore efficienza della rete ed affidabilità:* la quantità di dati che devono essere trasferiti attraverso la rete è ridotta: invece di inviare flussi di dati grezzi e massicci, i dispositivi edge possono pre-elaborare le informazioni e inviare al cloud solo i dati più significativi. Questo alleggerisce il carico sulla banda, migliorando l'efficienza della rete e riducendo i costi.
- *Scalabilità e flessibilità:* la rimozione, l'aggiunta e la gestione dei dispositivi che raccolgono i dati ambientali è semplificata con un'architettura basata sull'edge computing.

B. Backend

Il servizio backend si occupa di diverse mansioni:

- Ricevere i dati raccolti dai sensori attivi nei parchi e salvare tali informazioni all'interno del database;
- Esporre delle API REST per il frontend, attraverso cui vengono fornite le misure aggiornate e i dati storici all'utente.

Il backend è il fulcro centrale del sistema, e sfruttando diverse tecnologie, si occupa della comunicazione con gli altri componenti del sistema (facendo anche intermediario in alcuni casi). Comunica direttamente con:

- i dispositivi edge (per ricevere i dati);
- il database (per salvare e recuperare i dati)
- il browser, utilizzato dall'utente (per inviargli i dati)

C. Frontend

Tale componente si occupa di gestire il web server, mettendosi in attesa di richieste da parte degli utenti, per poi restituirgli le risorse web (HTML+Javascript+CSS).

Il frontend viene contattato dall'utente tramite browser, e poi la pagina web interagirà con il backend, tramite richieste periodiche, e mostrerà i dati in tempo reale, sia in forma testuale che tramite grafici.

D. Database

Il database è basato su MariaDB che è un sistema di gestione di database relazionali, e si occuperà di mantenere le informazioni raccolte dai dispositivi edge, e le informazioni necessarie al funzionamento del sistema.

E. Admin

Ci sarà un componente dedicato all'amministratore del sistema con il quale esso può inserire, rimuovere parchi e sensori, oltre ad effettuare l'associazione e la dissociazione dei sensori dai parchi.

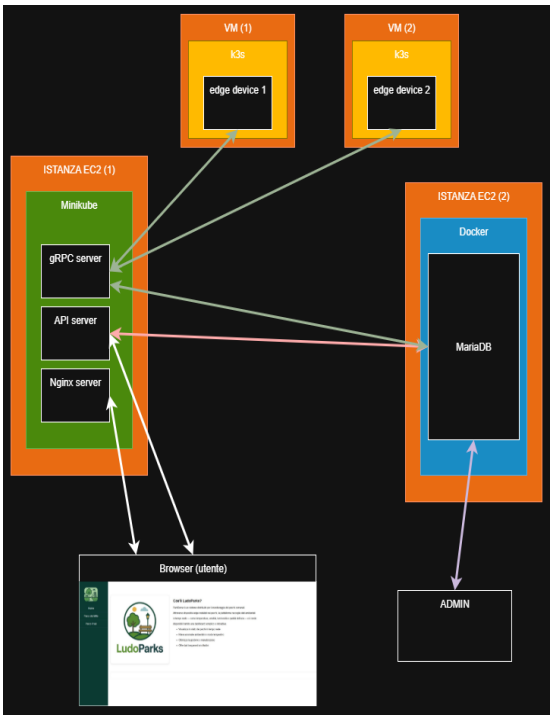


diagramma dell'architettura del sistema

V. IMPLEMENTATION

A. *Dispositivi edge*

Per rendere la simulazione più realistica possibile, i dispositivi edge girano su macchine virtuali (istanze EC2 di AWS), e sostanzialmente consisterebbero in dispositivi installati in parchi comunali. Sono realizzati come microservizi Python orchestrati con k3s.

L'utilizzo di k3s, nonostante ci sia un solo microservizio attivo comporta diversi vantaggi:

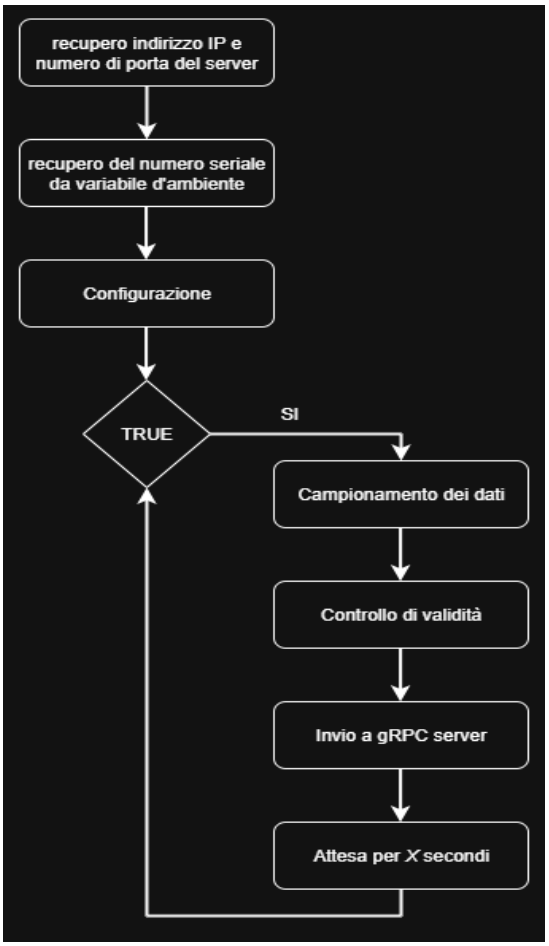
- Simulazione più realistica: nei dispositivi edge reali molto spesso è utilizzato k3s;
- Monitoraggio e logging integrato: i controlli sullo stato di salute del sistema sono supportati e automatizzati da k3s;
- Resilienza: riavvio automatico in caso di crash del container;
- Scalabilità futura: k3s facilita l'aggiunta di nuovi microservizi sullo stesso edge e la replicazione del servizio;
- Isolamento: il microservizio è in esecuzione in uno spazio virtualizzato nell'host;
- Portabilità: il deployment funziona identicamente su diversi dispositivi edge e ciò rende facilitata la migrazione e la replica dell'ambiente.

All'avvio della macchina virtuale, il cluster k3s viene resetato, e si avvia il microservizio Python. Inizialmente, il dispositivo recupera da un file di configurazione le informazioni necessarie per potersi collegare al server, ovvero l'indirizzo IP e il numero di porta; poi recupera dalla variabile d'ambiente SERIAL_NUMBER il numero seriale univoco associato al dispositivo, il quale è stato definito tramite un componente ConfigMap (che serve per dichiarare le variabili d'ambiente) del cluster k3s.

Fatto ciò, viene creata un'istanza della classe *Sensor*, nel cui costruttore avviene l'inizializzazione dei parametri e la configurazione del dispositivo edge: il dispositivo contatta il server cloud (tramite gRPC) inviandogli il proprio numero seriale (per segnalarsi come attivo) e gli verrà restituito l'id del parco a cui è associato, il proprio id (i quali serviranno successivamente per identificare le misure raccolte) e l'intervallo di tempo che intercorre tra una misurazione e la successiva.

Dopodiché si entra all'interno di un loop infinito, nel quale si ripetono le seguenti operazioni:

- *Recupero delle misure*: vengono generate delle misurazioni (tramite il modulo AdvancedSensorSimulator.py) e impacchettate, pronte per essere inviate.
- *Controllo validità*: i dati prima di essere inviati passano un controllo di validità, ovvero si verifica che i dati raccolti siano validi (non ci siano valori insensati), e che non siano ridondanti (la nuova misurazione deve esprimere un cambiamento significativo rispetto all'ultima misurazione), perché nel caso non fossero validi non verrebbero inviati, e ciò permetterebbe di risparmiare risorse del server.
- *Invio dei dati*: i dati sono validi per cui sono inviati al server tramite una comunicazione asincrona gRPC.



Edge flowchart

Il servizio, per poter girare su k3s, è stato prima containerizzato tramite Docker, per poi essere caricato su una repository di Docker Hub, perchè ciò facilita il recupero dell'immagine. L'ambiente su k3s è stato settato tramite un file di deployment (edge.yaml) che serve per generare il container dentro il cluster di k3s, e un file per il Config Map (edge-cm.yaml) che è servito per definire le variabili d'ambiente dentro il container.

B. Backend

Il backend rappresenta il nucleo logico dell'intero sistema distribuito. È stato sviluppato in linguaggio Go per la sua efficienza nella gestione concorrente e per la facilità di integrazione con servizi gRPC e RESTful. Le sue principali responsabilità sono la ricezione e gestione dei dati inviati dai dispositivi edge, il salvataggio delle misurazioni nel database relazionale e l'esposizione di API HTTP per l'interfaccia web.

Il backend è stato implementato con 2 microservizi distinti: uno per le comunicazioni con gli edge (gRPC server) ed uno per le comunicazioni con i browser (API server). Tale scelta è stata veicolata dalla volontà di rendere i due servizi indipendenti, così da aumentarne la disponibilità; infatti, se uno dei due microservizi fallisse, non avrebbe ripercussioni sull'altro. Ognuno dei due microservizi è eseguito all'interno di un pod nel cluster Minikube e sono resi disponibili all'esterno tramite un componente Service del cluster, di tipo NodePort (serve per associare le porte dei pod a quelle del container di Minikube).

gRPC server: questo microservizio si occupa di mettersi in ascolto sulla porta 50051, in attesa di messaggi da parte dei dispositivi edge, quindi sia messaggi per la configurazione, sia messaggi contenenti le misurazioni raccolte.

Se riceve un messaggio di configurazione, il server dovrà recuperare dal database l'id del sensore e del parco a cui è stato associato, ma prima di inviarglieli viene modificato lo status del sensore e del parco, i quali sono rispettivamente impostati come 'attivo' e 'osservato'.

Se invece, il messaggio ricevuto è l'insieme delle misurazioni raccolte dal dispositivo edge, alla ricezione viene avviata una goroutine per eseguire l'inserimento dei dati nel database: in questo modo il server non resta bloccato durante le operazioni di inserimento dei dati e riesce a gestire simultaneamente più dispositivi edge attivi, rendendo il sistema scalabile e performante.

API server: l'altro microservizio si occupa di esporre un set di API REST utilizzate dal browser per recuperare i dati in formato JSON.

Precisamente sono esposte due API sulla porta 8080:

- */api/hello:* per ogni parco viene recuperata la misurazione più recente di ogni metrica, e sono anche recuperate le informazioni di temperatura massima e minima dei sette giorni precedenti
- */api/getData:* serve per recuperare le misurazioni avvenute durante la giornata odierna per una specifica metrica, le quali saranno poi usate per realizzare un grafico

La logica interna delle API consiste nella lettura dei parametri di query, nell'esecuzione di interrogazioni SQL sul database e nella serializzazione dei risultati in formato JSON.

Uno degli aspetti fondamentali del backend è la capacità di gestire molteplici richieste contemporaneamente, provenienti da dispositivi

edge o dal frontend; questo è reso possibile dall'uso delle goroutine di Go, che permettono un'esecuzione concorrente e leggera. Ogni richiesta gRPC ricevuta avvia una nuova goroutine per il salvataggio dei dati, così come ogni richiesta REST viene servita in modo indipendente. Questo modello consente al backend di mantenere alte prestazioni e tempi di risposta ridotti anche in condizioni di carico.

Per quanto riguarda le operazioni CRUD, per evitare problemi di inconsistenza causati da fallimenti, esse sono state implementate come transizioni; per cui se una funzionalità che comprende più operazioni CRUD dovesse fallire durante la sua esecuzione, non si avrebbero problemi di inconsistenza.

C. Frontend

Il frontend dell'applicazione costituisce l'interfaccia grafica accessibile agli utenti e rappresenta il punto di visualizzazione dei dati ambientali raccolti dai dispositivi edge. È stato progettato per essere semplice, chiaro e responsive, offrendo una panoramica in tempo reale dello stato dei parchi comunali monitorati.

Il frontend è containerizzato all'interno di un'immagine Docker basata su un web server Nginx, che ha il compito di servire i file statici HTML, CSS e JavaScript. Questa immagine è poi eseguita come pod all'interno del cluster Minikube, esattamente come per i microservizi del backend.

Per rendere il frontend accessibile anche dall'esterno (es. per accedervi da browser su internet), è stato configurato un Service di tipo NodePort che instrada il traffico HTTP entrante nel nodo verso il pod contenente Nginx.

L'inclusione del server Nginx nel cluster si basa su una logica di coerenza architetturale: tutte le principali componenti dell'applicazione sono orchestrate all'interno dello stesso ambiente controllato, favorendo una gestione centralizzata e semplificata dell'infrastruttura.

Inoltre, mantenere il frontend all'interno del cluster rende più agevole il deployment dell'intera applicazione: il sistema può essere avviato tramite pochi comandi (kubectl apply) utilizzando file YAML, che definiscono le immagini per i pod, il loro numero e i componenti Service. Questa modalità semplifica l'automazione del rilascio e facilita la replicabilità dell'ambiente, sia in fase di sviluppo che in ambienti di test o produzione.

Dal punto di vista della scalabilità, anche il servizio web può beneficiare delle funzionalità offerte da Minikube. Qualora l'applicazione venisse utilizzata in un contesto reale con numerosi accessi simultanei, sarebbe possibile scalare automaticamente i pod che eseguono Nginx, garantendo una distribuzione efficiente del carico.

Utilizzando Minikube, di base i microservizi non sono accessibili da Internet, ma è necessario esporli. Il componente Service dentro il cluster Minikube si occupa di fornire un punto di accesso stabile e un bilanciamento del carico per i microservizi, però li espone sulle porte del container di Minikube (Minikube gira come un container poichè il driver usato è Docker) e non sulle porte dell'host. Quindi, per rendere accessibili i microservizi è necessario mappare le porte dell'host a quelle di Minikube: tramite il comando ***kubectl port-forward*** si può fare in modo che il traffic in ingresso su una porta venga automaticamente reindirizzato.

MICROSERVIZIO	PORTA HOST	PORTA MINIKUBE
gRPC server	31008	50051
API server	31003	8080
Web server	32009	80

Sulla macchina virtuale su cui gira Minikube, è stato inserito uno script che è eseguito all'avvio della istanza e che si occupa di creare e mantenere attiva la mappatura tra le porte dell'host e quelle di Minikube.

D. Database

Il sistema fa uso di un database relazionale per la persistenza dei dati raccolti dai dispositivi edge e delle informazioni relative ai parchi e ai sensori. Nello specifico, è stato adottato MariaDB, una scelta motivata dalla sua leggerezza, compatibilità con MySQL e semplicità di gestione. Il database è stato posizionato su una macchina virtuale EC2 dedicata, separata da quella che ospita Minikube e da quelle dei dispositivi edge.

All'interno di quest'istanza EC2, il database viene eseguito come contenitore Docker. Tale scelta è stata fatta per garantire maggiore isolamento, portabilità e facilità di deploy, sfruttando i vantaggi offerti dalla containerizzazione. Il backend, che si trova all'interno del cluster Minikube, comunica con il database esterno tramite il suo indirizzo IP

pubblico (Elastic IP) e una porta esplicitamente esposta dal container Docker.

Il modello relazionale è stato progettato per essere semplice ma efficace. Le tabelle sono:

- ***Parks:*** contiene le informazioni anagrafiche sui parchi monitorati (id, name, location, is_observed).
- ***Sensors:*** rappresenta i sensori installati nei parchi, con un riferimento al parco associato, lo stato di attivazione e un identificativo univoco (id, serial_number, is_active, park_id).
- ***Measures:*** registra tutte le rilevazioni dei sensori, con i valori delle metriche raccolte e il relativo timestamp (id, sensor_id, park_id, temperature, humidity, brightness, air_quality, timestamp).

Le operazioni di scrittura nel database sono effettuate dal backend in modo concorrente, ma sicuro, sfruttando le goroutine di Go. MariaDB, da parte sua, gestisce correttamente la concorrenza in lettura e scrittura tramite transazioni e lock automatici, garantendo la coerenza dei dati.

E. Admin

Per l'amministratore del sistema è disponibile un programma separato che gli consente di eseguire le seguenti operazioni:

- Mostrare i sensori registrati
- Mostrare i parchi registrati
- Aggiungere un sensore
- Aggiungere un parco
- Rimuovere un sensore
- Rimuovere un parco
- Associare un sensore ad un parco
- Disassociare un sensore da un parco

Le operazioni CRUD eseguite dall'amministratore sono eseguite come transizioni per evitare conflitti e problemi di inconsistenza.

VI. DISCUSSION

Il progetto realizzato rispecchia quella che era la richiesta, ovvero realizzare un sistema che sfruttasse l'edge computing, i microservizi e la tecnologia di Kubernetes, con le sue varianti.

Durante la realizzazione del progetto sono emersi diversi aspetti significativi, sia positivi che critici, che meritano una riflessione.

Uno dei principali punti di forza del sistema è la **modularità**. L'adozione di un'architettura microservizi ha permesso di separare logicamente le responsabilità dei diversi componenti, semplificando il testing, la manutenzione e il deployment. Inoltre, la containerizzazione di ciascun modulo ha facilitato lo sviluppo locale e la successiva integrazione in un cluster Kubernetes, simulando uno scenario produttivo realistico.

La scelta di utilizzare **gRPC per la comunicazione tra edge e backend** si è rivelata vincente in termini di leggerezza, efficienza e chiarezza nella definizione delle interfacce. Tuttavia, ha richiesto una fase iniziale di apprendimento non banale, soprattutto per la corretta gestione dei file .proto e la sincronizzazione dei client/server in linguaggi differenti (Python e Go).

Uno dei principali aspetti affrontati è la **gestione della concorrenza nel backend**, specialmente nel contesto dell'inserimento asincrono dei dati nel database. Il linguaggio di programmazione Go ha semplificato molto tale problema, poiché grazie alle goroutine gli accessi avvengono in maniera sequenziale non causando problemi.

D'altro canto, il progetto può anche essere migliorato sotto alcuni aspetti. La **sicurezza** delle comunicazioni, poiché non sono state implementate accortezze per garantire una comunicazione criptata e sicura. Poi il sistema potrebbe essere migrato su un vero **cluster Kubernetes** che consentirebbe di sfruttare a pieno i vantaggi offerti dalla distribuzione del sistema in più nodi.